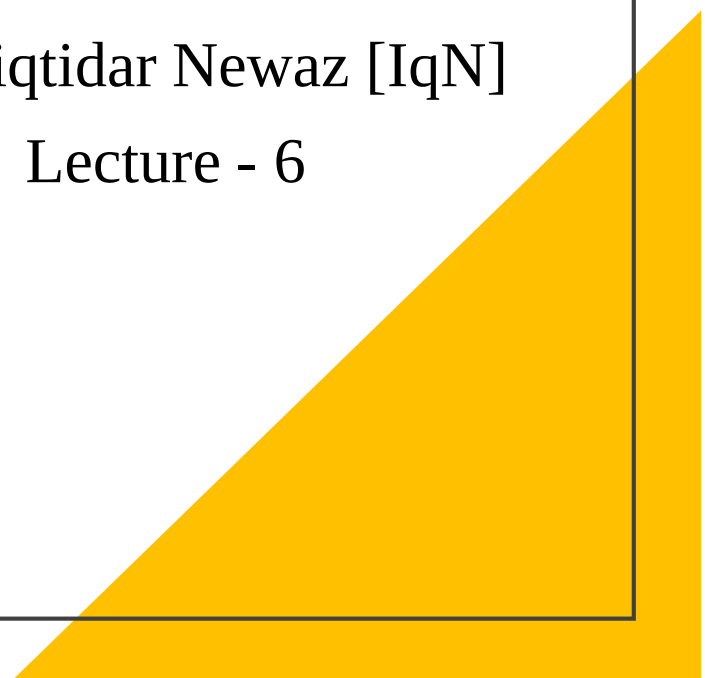


Class Diagram and Sequence Diagram

Lecture-6

AKM iqtidar Newaz [IqN]

Lecture - 6

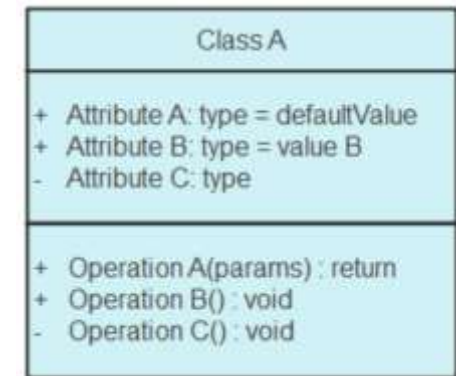
A large yellow triangle is positioned in the bottom right corner of the slide, pointing towards the top right.

Class Diagram

- What is a Class Diagram?

A class diagram is a static structure that is used in software engineering. A class diagram shows the classes, attributes, operations, and the relationship between them. This helps software engineers in developing the code for an application.

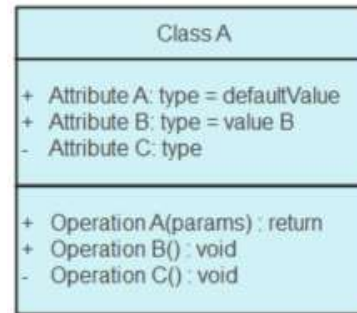
- Basic Components of Class Diagram:
 - Upper section: Contains the name of the class.
 - Middle section: Contains the attributes of the class.
 - Bottom section: Includes class operations (methods).Displayed in list format, each operation takes up its own line.



Class Interactions

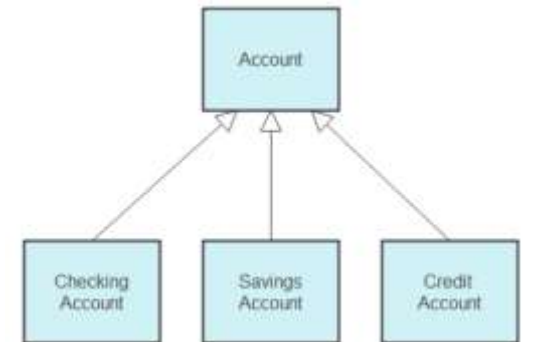
- **Member access modifiers**

- All classes have different access levels depending on the access modifier (visibility). Here are the access levels with their corresponding symbols:
- Public (+)
- Private (-)
- Protected (#)
- Package (~)
- Static (underlined)



Interactions between classes:

- **Inheritance:** The process of a child or sub-class taking on the functionality of a parent or superclass, also known as generalization.



- **Dependencies:** Dependency shows that one class depends on another. Change in one class will create change in another class. For example, an employee is dependent on the organization.

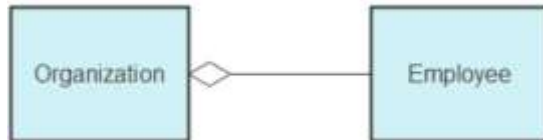


Class Interactions

- **Association:** Association shows a static relationship between two entities. The association between a student and school is “studies”.



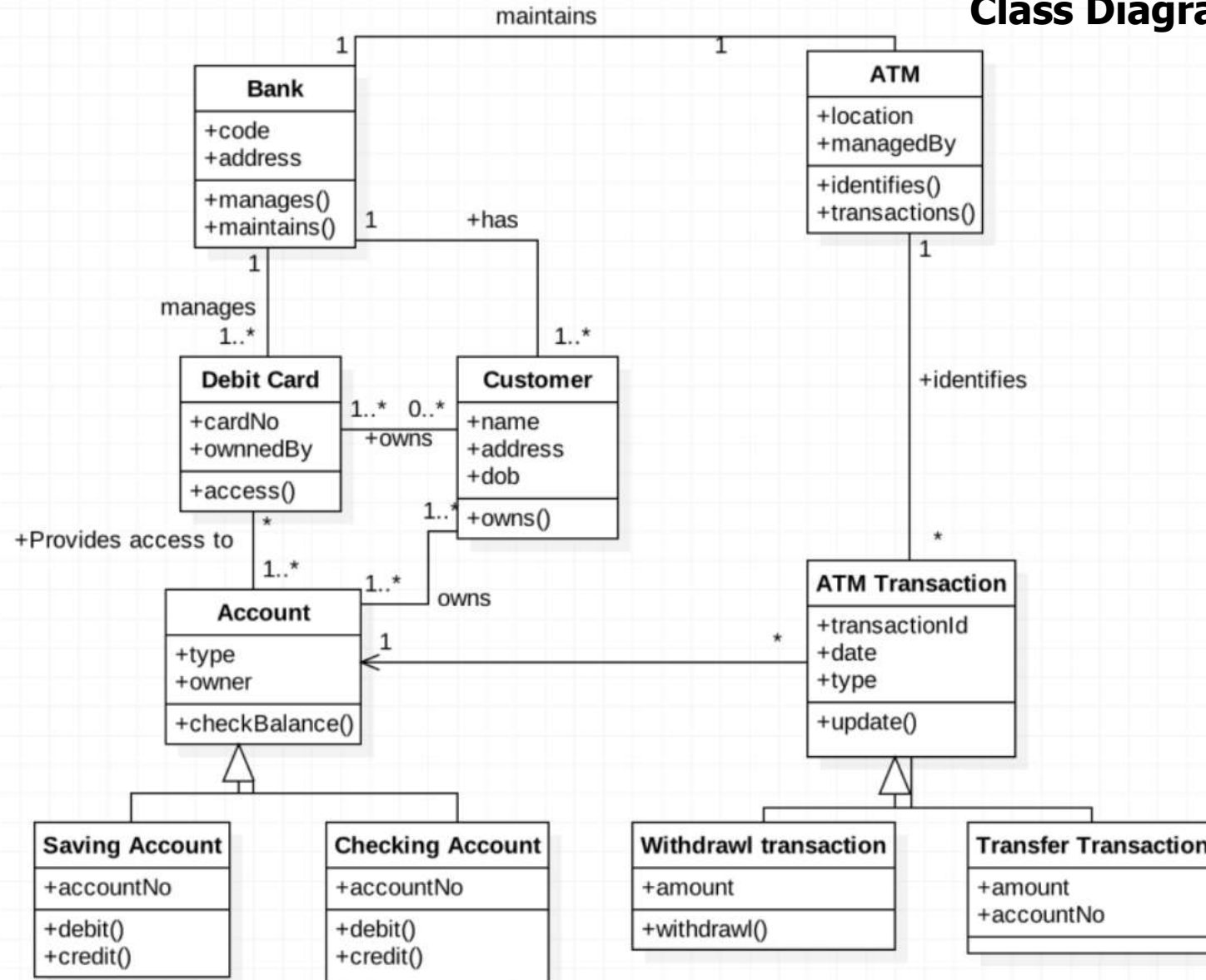
- **Aggregation:** In aggregation, 2 classes have a whole-part relationship. For example, if an employee does not come, the organization will remain there.



- The aggregation has another special type, called **composition**. In composition, a class is strongly connected to another class that it will stop functioning without it. For example, if an organization closes, all employees will have to leave.



Class Diagram Example: ATM

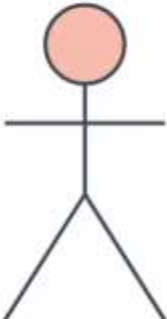


Sequence Diagram

- What is Sequence Diagram?

A sequence diagram is a type of interaction diagram because it describes how and in what order a group of objects works together. These diagrams are used by software developers and business professionals to understand requirements for a new system or to document an existing process. Sequence diagrams are sometimes known as event diagrams or event scenarios.

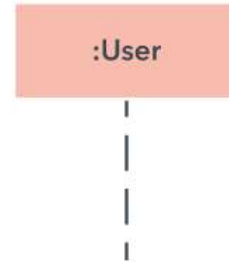
Basic Symbols and Components

Symbol	Name	Description
	Object symbol	Represents a class or object in UML. The object symbol demonstrates how an object will behave in the context of the system. Class attributes should not be listed in this shape.
	Activation box	Represents the time needed for an object to complete a task. The longer the task will take, the longer the activation box becomes.
	Actor symbol	Shows entities that interact with or are external to the system.



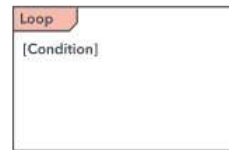
Package symbol

Used in UML 2.0 notation to contain interactive elements of the diagram. Also known as a frame, this rectangular shape has a small inner rectangle for labeling the diagram.



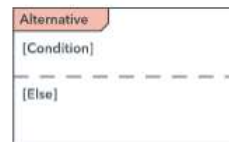
Lifeline symbol

Represents the passage of time as it extends downward. This dashed vertical line shows the sequential events that occur to an object during the charted process. Lifelines may begin with a labeled rectangle shape or an actor symbol.



Option loop symbol

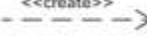
Used to model if/then scenarios, i.e., a circumstance that will only occur under certain conditions.



Alternative symbol

Symbolizes a choice (that is usually mutually exclusive) between two or more message sequences. To represent alternatives, use the labeled rectangle shape with a dashed line inside.

Sequence Diagram Example: ATM Transaction

Symbol	Name	Description
	Synchronous message symbol	Represented by a solid line with a solid arrowhead. This symbol is used when a sender must wait for a response to a message before it continues. The diagram should show both the call and the reply.
	Asynchronous message symbol	Represented by a solid line with a lined arrowhead. Asynchronous messages don't require a response before the sender continues. Only the call should be included in the diagram.
	Asynchronous return message symbol	Represented by a dashed line with a lined arrowhead.
	Asynchronous create message symbol	Represented by a dashed line with a lined arrowhead. This message creates a new object.
	Reply message symbol	Represented by a dashed line with a lined arrowhead, these messages are replies to calls.
	Delete message symbol	Represented by a solid line with a solid arrowhead, followed by an X. This message destroys an object.

